

Lecture 03

From Data to Deep Learning

Representation Learning, Function Composition, Layers and CNNs

Dinh Que Tran

Intelligent System Development
Academic Year 2026–2027

Lecture 03 – Roadmap

- ① From Traditional ML to Deep Learning
- ② Why Representation Matters
- ③ Data as Tensors
- ④ Functions and Parameterized Functions
- ⑤ Function Composition
- ⑥ From Functions to Layers
- ⑦ Numerical Neural Network
- ⑧ Deep Representation Learning
- ⑨ Three Applications:
 - Diabetes
 - House Price
 - Customer Comments
- ⑩ Learning: Loss, Gradient and Update
- ⑪ Why CNN for Images?
- ⑫ Numerical Convolution
- ⑬ CNN for Skin Disease

Where Did We Stop?

In Lectures 01–02, we studied the classical machine-learning workflow:

Raw Data → Feature Engineering → ML Model → Prediction

Examples:

- Diabetes: clinical measurements
- House price: numerical and categorical attributes
- Customer comments: text → TF-IDF

The model learns from the representation we give it.

Three Applications We Already Know

Problem		Raw Input	Representation
Diabetes		Patient measurements	Age, BMI, glucose, blood pressure, etc.
House price		House information	Area, bedrooms, location, age, etc.
Customer com-ments		Natural language	TF-IDF / word counts / engineered features

The difficult question is not only “Which model should we use?” but also “How should the data be represented?”

A New Question

Suppose the input is an image.

A raw image contains thousands or millions of pixel values.

$$X \in \mathbb{R}^{H \times W \times C}.$$

For a 224×224 RGB image:

$$224 \times 224 \times 3 = 150\,528$$

pixel values.

Ask students:

What features should a human engineer?

Possible features:

- edges?
- color distributions?
- textures?

The Representation Bottleneck

Traditional ML:

$$x \xrightarrow{\text{human design}} \phi(x) \xrightarrow{\text{ML}} y$$

where $\phi(x)$ is a manually designed representation.

Deep learning:

$$x \xrightarrow{f_1} h_1 \xrightarrow{f_2} h_2 \xrightarrow{f_3} \dots \xrightarrow{f_L} \hat{y}.$$

Now the intermediate representations

$$h_1, h_2, \dots, h_{L-1}$$

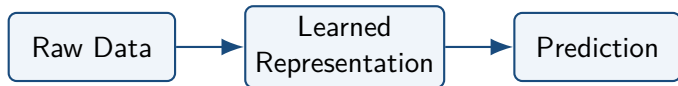
are **learned from data**.

Deep Learning = Learning representations together with the predictive function.

The Central Transition



Traditional ML



Deep Learning

Traditional ML: representation is largely designed first.
Deep Learning: representation is learned jointly with the task.

From Scalar to Tensor

Scalar

$$x = 5.$$

Vector

$$\mathbf{x} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix} \in \mathbb{R}^3.$$

Matrix

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

Tensor

A general multidimensional numerical array:

Image as a Tensor

A color image can be represented as

$$X \in \mathbb{R}^{H \times W \times C}.$$

For RGB:

$$C = 3.$$

Therefore:

$$X \in \mathbb{R}^{224 \times 224 \times 3}.$$

Each value represents a pixel intensity:

$$X_{i,j,c}.$$

where:

i : vertical position

Different Problems – Different Tensor Structures

Problem	Typical Representation	Example Shape
Diabetes	Feature vector	(d)
House price	Feature vector	(d)
Customer comments	Token/embedding sequence	(T, d)
Image	Spatial tensor	(H, W, C)
Batch of images	Batch tensor	(B, C, H, W)

The tensor shape describes how information is organized.

Start With a Function

Consider:

$$f(x) = 2x + 1.$$

For

$$x = 3,$$

we obtain

$$f(3) = 2(3) + 1 = 7.$$

Now introduce parameters:

$$f_{\theta}(x) = wx + b.$$

where

Function Composition

Suppose

$$f_1(x) = 2x + 1$$

and

$$f_2(x) = 3x - 2.$$

Define

$$F(x) = f_2(f_1(x)).$$

For $x = 4$:

$$f_1(4) = 2(4) + 1 = 9.$$

Then

$$f_2(9) = 3(9) - 2 = 25$$

Function Composition as a Network



Mathematically:

$$F = f_2 \circ f_1$$

and more generally:

$$F = f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

A neural network is fundamentally a composition of parameterized transformations.

Why Nonlinearity?

Suppose every layer is linear:

$$f_1(x) = W_1x + b_1$$

and

$$f_2(x) = W_2x + b_2.$$

Then

$$f_2(f_1(x)) = W_2(W_1x + b_1) + b_2.$$

Therefore:

$$f_2(f_1(x)) = (W_2W_1)x + (W_2b_1 + b_2).$$

This is still just:

$$Wx + b$$

Activation Function

Introduce a nonlinear activation:

$$h = \sigma(Wx + b).$$

A very common activation is ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Examples:

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(2.5) = 2.5.$$

Now:

$$F(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

What Is a Layer?

A layer is a parameterized transformation:

$$h = f_{\theta}(x).$$

For a dense layer:

$$h = \sigma(Wx + b)$$

where:

- x : input representation
- W : learnable weights
- b : learnable bias
- σ : nonlinear activation
- h : new representation

Therefore a layer does not merely “store neurons”.

A layer transforms one representation into another.

A Small Neural Network

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y}$$

with

$$h_1 = \text{ReLU}(W_1x + b_1),$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2),$$

and

$$\hat{y} = \text{softmax}(W_3h_2 + b_3).$$

Therefore:

$$\boxed{\hat{y} = f_3(f_2(f_1(x)))}$$

This is the basic mathematical structure of a feed-forward deep neural network.

Numerical Example – Input

Let $x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

First layer: $W_1 = \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix}$ and $b_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

Calculate:

$$z_1 = W_1 x + b_1.$$

$$= \begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Numerical Example – First Layer

Compute each component: $z_{1,1} = 1(2) + 2(1) + 0 = 4$ $z_{1,2} = 2(2) - 1(1) + 1 = 4$.

Therefore: $z_1 = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$

Apply ReLU: $h_1 = \text{ReLU}(z_1) = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$.

One layer has transformed x into a new representation h_1 .

Numerical Example – Second Layer

Let $W_2 = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}$, $b_2 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$.

Then $z_2 = W_2 h_1 + b_2$. Therefore:

$$\begin{aligned} z_2 &= \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 4 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1 \\ 12 \end{bmatrix}. \end{aligned}$$

So:

$$z_2 = \begin{bmatrix} 1 \\ 12 \end{bmatrix}.$$

Numerical Example – Softmax

For logits: $z = \begin{bmatrix} 1 \\ 12 \end{bmatrix}$, softmax is $P(y = k \mid x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$.

Thus:

$$P_1 = \frac{e^1}{e^1 + e^{12}} \quad P_2 = \frac{e^{12}}{e^1 + e^{12}}.$$

Approximately:

$P_1 \approx 0.0000167$	$P_2 \approx 0.9999833$
-------------------------	-------------------------

The network strongly prefers class 2.

Application 1 – Diabetes

Suppose we want to predict whether a patient has diabetes.

A simplified feature vector:

$$x = \begin{bmatrix} \text{glucose} \\ \text{BMI} \\ \text{age} \end{bmatrix}.$$

Example patient:

$$x = \begin{bmatrix} 150 \\ 32 \\ 50 \end{bmatrix}.$$

Traditional ML:

$$x \rightarrow \text{classifier} \rightarrow y.$$

Deep learning:

Diabetes – Normalize the Input

Raw features have different scales.

Example:

$$\text{glucose} = 150, \quad \text{BMI} = 32, \quad \text{age} = 50.$$

A common preprocessing idea is standardization:

$$z = \frac{x - \mu}{\sigma}.$$

Suppose:

$$\mu_{\text{glucose}} = 120, \quad \sigma_{\text{glucose}} = 30.$$

Then:

$$z_{\text{glucose}} = \frac{150 - 120}{30} = 1.$$

Suppose:

Diabetes – A Small Neural Model

After normalization, suppose

$$x = \begin{bmatrix} 1 \\ 0.4 \\ 0.5 \end{bmatrix}.$$

Consider one neuron:

$$z = w^T x + b.$$

Let

$$w = \begin{bmatrix} 0.8 \\ 0.5 \\ 0.2 \end{bmatrix}, \quad b = -0.6.$$

Then:

Diabetes – What Did the Network Learn?

The model does not simply memorize:

glucose $\rightarrow$ diabetes.

It learns a function:

$$f_{\theta} : \mathbb{R}^d \rightarrow [0, 1].$$

For multiple layers:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow p.$$

For example:

$$h_1 = \text{ReLU}(W_1x + b_1)$$

$$h_2 = \text{ReLU}(W_2h_1 + b_2)$$

Application 2 – House Price

Now consider regression.

Input:

$$x = \begin{bmatrix} \text{area} \\ \text{bedrooms} \\ \text{age} \\ \text{distance to center} \end{bmatrix}.$$

Output:

$\hat{y}$ = predicted house price.

Traditional ML:

$$\hat{y} = w^T x + b.$$

Deep learning:

House Price – A Numerical Example

Suppose normalized features are:

$$x = \begin{bmatrix} 1.2 \\ 0.5 \\ -0.3 \\ -0.8 \end{bmatrix}.$$

A hidden neuron has

$$w = \begin{bmatrix} 0.5 \\ 0.8 \\ -0.4 \\ 0.2 \end{bmatrix}, \quad b = 0.1.$$

Then:

$$z = w^T x + b.$$

Calculate:

House Price – Several Hidden Features

Suppose the hidden layer produces:

$$h = \begin{bmatrix} 1.06 \\ 0.20 \\ 0 \\ 0.75 \end{bmatrix}.$$

The next layer can combine these learned representations:

$$z_2 = w^T h + b.$$

For example:

$$w = \begin{bmatrix} 0.7 \\ 0.2 \\ -0.1 \\ 0.5 \end{bmatrix}, \quad b = 0.3.$$

Then:

House Price – Regression Output

For regression, the final output can simply be:

$$\hat{y} = w^T h + b.$$

Suppose:

$$\hat{y} = 2.35.$$

If the target was normalized, we may transform it back to the original price scale.

For example:

$$y_{\text{real}} = \mu_y + \sigma_y \hat{y}.$$

If:

$$\mu_y = 300\,000, \quad \sigma_y = 100\,000,$$

then:

House Price – What Changed?

The task is still:

$$x \rightarrow y.$$

But deep learning inserts learned representations:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \hat{y}.$$

The important idea is not:

“deep learning = image classification”

but:

deep learning = learned composition of transformations

This applies to:

- regression
- classification
- text

Application 3 – Customer Comments

Consider an e-commerce comment:

"The phone is beautiful but the battery is terrible."

Possible tasks:

- sentiment classification
- product aspect detection
- complaint detection
- customer-interest prediction
- review quality classification

Traditional ML might use:

text $\rightarrow$ TF-IDF $\rightarrow$ classifier.

Deep learning:

text $\rightarrow$ learned representation $\rightarrow$ prediction

Customer Comment – TF-IDF Representation

Suppose our vocabulary is:

[beautiful, battery, terrible, phone].

For the sentence:

“beautiful battery terrible phone”

a simple count vector could be:

$$x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

With TF-IDF, the values are weighted according to term frequency and document frequency.
The result is still:

The Limitation of a Bag-of-Words Representation

Compare:

“The product is good.”

and

“The product is not good.”

A simple bag-of-words representation may contain almost the same words.
But the meaning is different.

Therefore:

Representation must capture relationships, not only individual words.

Deep learning can learn representations in which words, tokens and their contexts are encoded.

From Words to Vectors

Suppose a vocabulary contains four words.

Assign each word a learned vector:

$$\text{good} \rightarrow \begin{bmatrix} 0.8 \\ 0.2 \\ 0.7 \end{bmatrix}$$

$$\text{bad} \rightarrow \begin{bmatrix} -0.7 \\ -0.1 \\ -0.6 \end{bmatrix}$$

$$\text{beautiful} \rightarrow \begin{bmatrix} 0.7 \\ 0.4 \\ 0.8 \end{bmatrix} .$$

These are examples of **learned representations**.

Instead of representing a word by one symbolic ID:

Customer Comment – Sequence Representation

A comment becomes a sequence:

$$(w_1, w_2, \dots, w_T).$$

Each token is mapped to a vector:

$$w_t \rightarrow e_t \in \mathbb{R}^d.$$

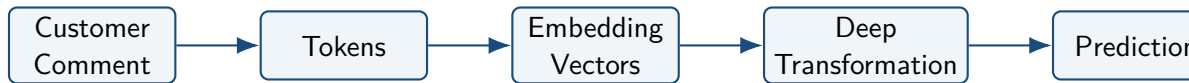
Therefore:

$$(w_1, \dots, w_T) \rightarrow (e_1, \dots, e_T).$$

Stacking vectors:

$$X = \begin{bmatrix} e_1^T \\ e_2^T \\ \vdots \\ e_T^T \end{bmatrix} \in \mathbb{R}^{T \times d}.$$

A Conceptual Text Network



Depending on the architecture, the deep transformation may involve:

- CNN
- RNN / LSTM
- Transformer
- Attention

These will be studied later.

What Happens Inside the Hidden Layers?

Suppose:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow y.$$

The representations can become increasingly task-specific:

$$x \rightarrow \underbrace{h_1}_{\text{low-level}} \rightarrow \underbrace{h_2}_{\text{intermediate}} \rightarrow \underbrace{h_3}_{\text{high-level}} \rightarrow y.$$

For images:

$$\text{pixels} \rightarrow \text{edges} \rightarrow \text{textures} \rightarrow \text{parts} \rightarrow \text{semantic patterns}.$$

For text:

$$\text{tokens} \rightarrow \text{local patterns} \rightarrow \text{context} \rightarrow \text{semantic representation}.$$

Representation as a Transformation

Let

$$h_1 = f_1(x).$$

Then

$$h_2 = f_2(h_1) = f_2(f_1(x)).$$

Then:

$$h_3 = f_3(h_2).$$

Therefore:

$$h_3 = f_3(f_2(f_1(x))).$$

Each layer receives a representation and produces another representation.

Deep learning can be understood as a sequence of learned coordinate

Why Multiple Representations?

Suppose a raw input contains:

10 000

numerical values.

The network might transform it into:

512

features, then:

128,

then:

32,

then:

Prediction Is Not Learning

Suppose the network computes:

$$\hat{y} = F_{\theta}(x).$$

This is only the **forward computation**.

To learn, we need a measure of error:

$$\mathcal{L}(y, \hat{y}).$$

Examples:

Classification:

$$\mathcal{L}_{\text{CE}} = - \sum_k y_k \log \hat{y}_k.$$

Regression:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum (y_i - \hat{y}_i)^2.$$

The Training Objective

Given training examples:

$$\{(x_i, y_i)\}_{i=1}^N,$$

the network is:

$$\hat{y}_i = F_{\theta}(x_i).$$

We seek:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, F_{\theta}(x_i))$$

This equation is extremely important.

It says:

Find parameters that minimize prediction error.

Forward $\rightarrow$ Loss $\rightarrow$ Gradient

Training consists conceptually of three stages.



Then:

$$\mathcal{L} \rightarrow \nabla_{\theta} \mathcal{L} \rightarrow \theta_{\text{new}}.$$

Gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

where η is the learning rate.

A One-Parameter Numerical Example

Suppose:

$$L(w) = (w - 3)^2.$$

Derivative:

$$\frac{dL}{dw} = 2(w - 3).$$

Start with:

$$w_0 = 1.$$

Then:

$$\left. \frac{dL}{dw} \right|_{w=1} = 2(1 - 3) = -4.$$

Let:

What Does Deep Mean?

Deep does not primarily mean:

- a large dataset;
- a large input;
- a complicated mathematical formula;
- many parameters.

Conceptually:

Deep $\approx$ many successive learned transformations

For example:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow y.$$

Each transformation can change the representation.

Shallow vs Deep

A simple model:

$$x \rightarrow y.$$

A shallow neural model:

$$x \rightarrow h_1 \rightarrow y.$$

A deeper model:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow y.$$

The deeper model can construct a hierarchy:

simple representations $\rightarrow$ composed representations $\rightarrow$ complex representations

This is the conceptual motivation for deep architectures.

Why Not Just Use a Fully Connected Network for Images?

An RGB image:

$$224 \times 224 \times 3$$

contains:

$$150\,528$$

values.

Suppose we flatten it and connect to a hidden layer of 1024 neurons.

Number of weights:

$$150\,528 \times 1024$$

$$\approx 154\,140\,672.$$

That is approximately:

Images Have Local Structure

Nearby pixels are related.

For example, a small region may contain:

edge

or

corner

or

texture.

Instead of connecting every pixel independently to every neuron, CNNs use local receptive fields.

A CNN is a deep network designed to exploit spatial structure.

Convolution – Numerical Example

Consider:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and kernel:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

At the upper-left position:

$$\begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}.$$

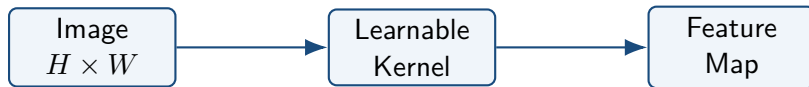
Element-wise multiplication:

Convolution Produces a Feature Map

The kernel is applied at multiple positions:

$$X \xrightarrow{\text{kernel}} \text{feature map.}$$

Conceptually:



The same kernel is reused at different spatial positions.

This gives CNNs:

- local connectivity;
- parameter sharing;
- translation-aware pattern detection.

CNN as Function Composition

A CNN still follows the same fundamental principle:

$$F = f_L \circ \cdots \circ f_2 \circ f_1$$

But now some transformations are convolutional:

$$h_1 = \text{ReLU}(\text{Conv}_{\theta_1}(x))$$

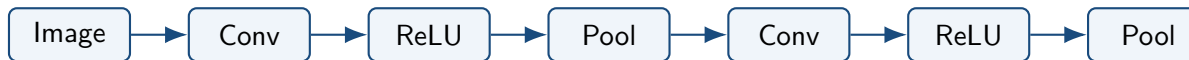
$$h_2 = \text{ReLU}(\text{Conv}_{\theta_2}(h_1)).$$

Finally:

$$\hat{y} = \text{softmax}(Wh_L + b).$$

CNN is not a completely different idea. It is a specialized deep representation-learning architecture.

Typical CNN Pipeline



Repeat transformations → obtain feature maps → classification head.

Image → Features → Representation → Prediction

Final Application – Skin Disease Classification

Suppose we have:

$$224 \times 224 \times 3$$

dermoscopic images.

Goal:

image $\rightarrow$ skin disease class.

For example:

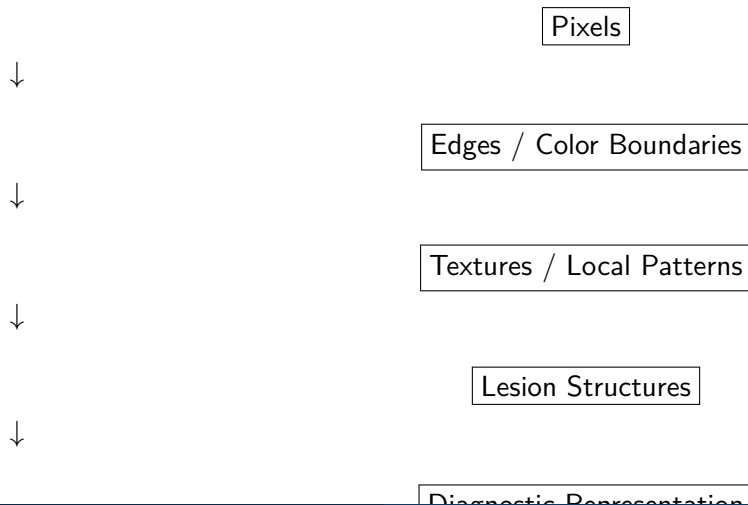
$\{\text{akiec}, \text{bcc}, \text{bkl}, \text{df}, \text{mel}, \text{nv}, \text{vasc}\}.$

The challenge:

- complex visual patterns;
- texture;
- color;
- shape;

CNN Representation Hierarchy for Skin Images

A conceptual hierarchy:



CNN Example – Tensor Dimensions

A conceptual CNN could transform:

$$224 \times 224 \times 3$$

into:

$$112 \times 112 \times 32$$

then:

$$56 \times 56 \times 64$$

then:

$$28 \times 28 \times 128$$

then:

$$14 \times 14 \times 256$$

CNN Classification – Mathematical View

Let:

$$x \in \mathbb{R}^{224 \times 224 \times 3}.$$

A CNN computes:

$$h_1 = f_1(x)$$

$$h_2 = f_2(h_1)$$

...

$$h_L = f_L(h_{L-1}).$$

Then:

One Idea – Three Different Data Types

Application	Input	Learned Transformation
Diabetes	Vector	Dense layers / MLP
House price	Vector	Dense layers / MLP
Customer com-ments	Token sequence	Embedding + sequence/context model
Skin image	Image tensor	Convolutional layers

But all can be expressed as:

$$\hat{y} = F_{\theta}(x)$$

and

$$F_{\theta} = f_L \circ \dots \circ f_1.$$

The Same Learning Principle

Diabetes

patient data $\rightarrow$ learned representation $\rightarrow P(\text{diabetes})$

House price

house attributes $\rightarrow$ learned representation $\rightarrow$ price

Customer comments

text $\rightarrow$ learned representation $\rightarrow$ sentiment / intent

Skin image

pixels $\rightarrow$ learned representation $\rightarrow$ disease.

The data modality changes. The fundamental learning principle remains.

Traditional ML vs Deep Learning

Traditional ML	Deep Learning
Feature engineering is important Often effective with structured data	Representation learning is central Especially powerful for complex unstructured data
Features often designed separately Models may be relatively shallow TF-IDF for text is common	Features and predictor learned jointly Multiple transformations are composed Embeddings/contextual representations are common
Hand-designed image features may be used	CNNs learn spatial representations

This does not mean traditional ML is obsolete.

Both approaches remain useful.

A More Precise Comparison

Traditional pipeline:

$$x \rightarrow \phi_{\text{human}}(x) \rightarrow g_{\theta} \rightarrow y$$

Deep-learning pipeline:

$$x \rightarrow f_{\theta_1} \rightarrow f_{\theta_2} \rightarrow \cdots \rightarrow f_{\theta_L} \rightarrow y$$

The key conceptual difference:

Where does the representation come from?

This question will appear repeatedly throughout the course.

From Model to Intelligent System

A deep-learning model alone is not necessarily an intelligent system.

A practical system may contain:

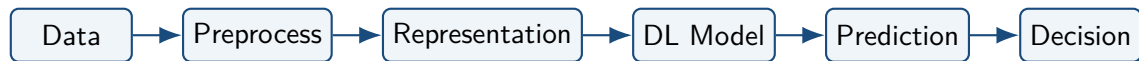
Data → Preprocessing → Representation → Model → Prediction → Decision → Action

Examples:

- diabetes screening;
- house-price recommendation;
- customer-service automation;
- medical-image assistance.

Intelligent System Development = model + data + software + decision + deployment.

A Deep-Learning System Pipeline



Later lectures will ask:

- How do we implement the model?
- How do we train it?
- How do we evaluate it?
- How do we deploy it?
- How do we monitor it?

Exercise 1 – Diabetes

Suppose a patient has:

$$x = \begin{bmatrix} 1.2 \\ 0.5 \\ -0.3 \end{bmatrix}.$$

A neuron has:

$$w = \begin{bmatrix} 0.5 \\ 0.8 \\ -0.4 \end{bmatrix}, \quad b = -0.2.$$

Calculate:

$$z = w^T x + b.$$

Then calculate:

Exercise 1 – Diabetes

Suppose a patient has:

$$x = \begin{bmatrix} 1.2 \\ 0.5 \\ -0.3 \end{bmatrix}.$$

A neuron has:

$$w = \begin{bmatrix} 0.5 \\ 0.8 \\ -0.4 \end{bmatrix}, \quad b = -0.2.$$

Calculate:

$$z = w^T x + b.$$

Then calculate:

Exercise 2 – House Price

Suppose:

$$x = \begin{bmatrix} 1 \\ 0.5 \\ -1 \end{bmatrix}.$$

Let:

$$w = \begin{bmatrix} 0.6 \\ 0.4 \\ 0.8 \end{bmatrix}, \quad b = 0.1.$$

Calculate:

$$z = w^T x + b.$$

Then apply ReLU.

Exercise 2 – House Price

Suppose:

$$x = \begin{bmatrix} 1 \\ 0.5 \\ -1 \end{bmatrix}.$$

Let:

$$w = \begin{bmatrix} 0.6 \\ 0.4 \\ 0.8 \end{bmatrix}, \quad b = 0.1.$$

Calculate:

$$z = w^T x + b.$$

Then apply ReLU.

Exercise 3 – Customer Comment

Consider:

"The camera is excellent but the battery is poor."

Ask:

- ① What is the raw input?
- ② How would TF-IDF represent it?
- ③ What information might TF-IDF lose?
- ④ Why might word embeddings help?
- ⑤ Why might context matter?
- ⑥ What type of deep model could process the sequence?

Possible answers:

Embedding → CNN/RNN/Transformer → representation → prediction.

Exercise 4 – Convolution

Given:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

calculate the convolution output using stride 1 and no padding.
Students should calculate all four positions.

Exercise 4 – Convolution

Given:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

and

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

calculate the convolution output using stride 1 and no padding.

Students should calculate all four positions.

Expected result:

$$\begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

“Deep Learning means a very complicated model.”

Not necessarily.

The conceptual definition is:

$$F = f_L \circ \cdots \circ f_1.$$

The important property is the composition of transformations.

A small network can illustrate deep-learning principles.

A large network can still be badly designed.

“Deep Learning automatically understands the data.”

No.

The model learns from:

data + architecture + loss + optimization.

Poor data can produce poor models.

Poor labels can produce poor models.

Poor preprocessing can produce poor models.

Poor architecture can produce poor models.

Deep learning reduces manual feature engineering; it does not eliminate engineering.

“More layers always give a better model.”

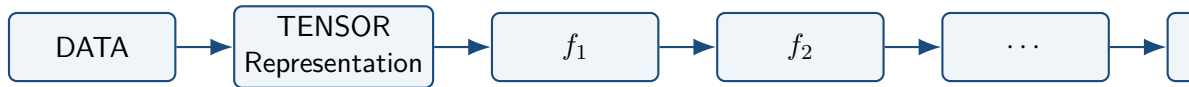
Not necessarily.

Increasing depth may introduce:

- optimization difficulty;
- overfitting;
- computational cost;
- instability;
- unnecessary complexity.

Architecture design is part of intelligent-system development.

The Big Picture



$$\hat{y} = F_{\theta}(x) = (f_L \circ f_{L-1} \circ \dots \circ f_1)(x)$$

Training:

$$\theta^* = \arg \min_{\theta} \sum_i \mathcal{L}(y_i, F_{\theta}(x_i))$$

From Features to Representations

Traditional ML:

$$x \rightarrow \boxed{\text{designed features}} \rightarrow \text{model} \rightarrow y$$

Deep Learning:

$$x \rightarrow \boxed{\text{learned representations}} \rightarrow \text{prediction}$$

$$\hat{y} = (f_L \circ \cdots \circ f_2 \circ f_1)(x)$$

Deep Learning is learned function composition.

Each function transforms a representation.

What is the representation?

For every intelligent system, ask:

How is the data represented?

Lecture 03 – Final Review

Students should now be able to explain:

- 1 What deep learning means.
- 2 Why representation learning matters.
- 3 What a tensor represents.
- 4 What a parameterized function is.
- 5 What function composition means.
- 6 Why nonlinear activation functions are needed.
- 7 What a neural-network layer does.
- 8 How parameters are learned through loss and gradients.
- 9 Why CNNs are suitable for images.
- 10 How diabetes, house-price, text and image problems can all be viewed through the same framework.

Next: building, training and evaluating neural networks in practice.

References

-  F. Chollet, *Deep Learning with Python*, 2nd Edition, Manning, 2021.
-  A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd Edition, O'Reilly, 2022.
-  I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
-  Y. LeCun, Y. Bengio, G. Hinton, “Deep Learning,” *Nature*, 521, 436–444, 2015.
-  A. Krizhevsky, I. Sutskever, G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *NeurIPS*, 2012.

Thank You

Questions and Discussion

Data → Representation → Function → Composition → Learning → Intelligent System

Dinh Que Tran

Intelligent System Development

2026–2027